PageRank Algorithm

Nima Shadab (358061), Riym Chaoui Aziz(352283)

1 Problem Description

The PageRank algorithm is used by Google to establish a ranking among all pages composing the web. It's based on the fact that each web can be represented by a directed graph G = (V, E), where each node N represents a page, and every edge E symbolises a link from one page to another. For algorithmic purposes, G is translated into its corresponding column-stochastic adjacency matrix $A \in \mathbb{R}^{n,n}$, from which we derive all pages' scores. This means that we are trying to compute each unkown (a page's score) based on other unknowns (all other pages' scores).

Mathematically speaking, this translates into computing the principal eigenvector, i.e. the eigenvector associated to the maximal eigenvalue. To do so, we make use of the Power Method that, conveniently, computes exactly the eigenvector we need.

2 Power Method Algorithm

The Power Method algorithm is an iterative method that computes the dominant eigenvalue, and its respective eigenvetor, of a generic square matrix. Since the matrix to be examined is column-stochastic, the *Perron eigenvector*, associated to $\lambda = 1$ can be used to rank the web pages.

Algorithm 1: Naive Power Method

Data: Adjacency matrix $A \in \mathbb{R}^{n \times n}$, max iterations max_iter
Result: Rank vector $\mathbf{x}_k$
$\mathbf{x}_k \leftarrow \mathbf{1}_{n \times 1}$; `/* Initialize with ones */`
while $max_iter > 0$ **do**
  $\mathbf{x}_k \leftarrow A\mathbf{x}_k$; `/* Matrix-vector product */`
  $\mathbf{x}_k \leftarrow \mathbf{x}_k / \sum x_k$; `/* Scaling result */`
  $max_iter \leftarrow max_iter - 1$;
end
return $\mathbf{x}_k$;

The Naive implementation of the Power Method uses a simplistic condition as its stopping criterion, it sets a maximum number of iterations. More advanced versions of this algorithm, which will be discussed below, will introduce an additional condition related to the absolute value of the difference between two successive approximations of the eigenvector.

3 Adaptation to Google's PageRank

When applied to matrices representing real-world like webs, the Naive Power Method fails or doesn't work correctly due to some conditions that are necessary for the algorithm to work. More specifically, the Power Method is defined for matrices whose eigenvalues can be ordered as follows

$$|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|$$

and more often than not, due to their non-strong connectivity, adjacency matrices representing webs may have multiple dominant eigenvalues, making more rankings possible. This issue is solved using a *teleportation matrix* and a *damping factor*.

Another problem arises when the web presents *dangling nodes*, yielding a *column-substochastic* matrix that can no longer be a representation of the probability of navigating from one page to another.

3.1 Teleportation

To ensure the uniqueness of the dominant eigenvalue λ_1, as stated by the *Perron-Frobenius Theorem*, we need to make the matrix *primitive*, thus guaranteeing to have a single dominant eigenvalue $\lambda_1 = 1$.

Algorithm 2: Advanced Power Method (Dense Implementation)

Data: Stochastic matrix $A \in \mathbb{R}^{n \times n}$, max iterations max_iter, damping factor m, tolerance tol
Result: Final rank vector $\mathbf{x}_k$
$n \leftarrow$ number of rows in A
$\mathbf{x}_k \leftarrow \mathbf{1}_{n \times 1}$ `/* Initialize rank vector with ones */`
$eig \leftarrow \infty$
$eig_{old} \leftarrow -\infty$
while $max_iter > 0$ ***and*** $|eig - eig_{old}| > tol$ **do**
  $eig_{old} \leftarrow eig$
  `/* Apply damping factor and teleportation */`
  $\mathbf{x}_k \leftarrow (1 - m)(A\mathbf{x}_k) + m\frac{\mathbf{1}}{\mathbf{n}}$
  `/* Normalize the vector to ensure it remains a probability distribution */`
  $\mathbf{x}_k \leftarrow \mathbf{x}_k / \sum x_k$
  `/* Update eigenvalue estimate using Rayleigh-like quotient */`
  $eig \leftarrow \mathbf{x}_k^T A \mathbf{x}_k$
  $max_iter \leftarrow max_iter - 1$
end
return $\mathbf{x}_k$

3.2 Dangling Nodes

When the modified representation matrix M is built, the columns corresponding to pages with no outlinks will be made of m/n entries, so the column-wise sum will be equal to m/n. Since we have chosen the value of m equal to 0.15, m/n is strictly smaller than 1, thus yielding a column *sub*stochastic matrix. Our objective is to construct a modified version of A that is column-stochastic but that at the same time does not change the degree of importance of the pages forming the web.

This is done by forcing dangling nodes to vote for all other pages. The relative rankings are not affected as all pages receive equal benefits.

Algorithm 3: Dangling Node Correction (Dense Implementation)

Data: Adjacency matrix $A \in \mathbb{R}^{n \times n}$
Result: Stochastic matrix $\hat{A}$ with no dangling nodes
$D \leftarrow \{j \mid \sum_{i=1}^{n} A_{ij} = 0\}$ `/* Identify indices of dangling nodes */`
$\hat{A} \leftarrow A$ `/* Create a copy of the original matrix */`
foreach $j \in D$ **do**
  `/* Replace zero columns with a uniform probability distribution */`
  for $i \leftarrow 1$ **to** n **do**
    $\hat{A}_{ij} \leftarrow \frac{1}{n}$
  end
end
return $\hat{A}$

4 Dense vs Sparse Representation

With the modifications reported above, the Power Method Algorithm is now ready to be tested. In real life cases, however, webs are made of extremely large numbers of pages, and computing the ranking for large scale matrices becomes extremely computationally expensive. Fortunately, we are able to exploit the fact that most pages within the web contain only a minuscule number of links to other pages, so we can use the *Compressed Sparse Row* format to represent our matrix. By using the *CSR* format, we are able to reduce our initial storage cost down to $O(V + E)$ from a previous point of $O(V^2)$.

Algorithm 4: CSR Representation

Data: Dense matrix $A \in \mathbb{R}^{m \times n}$
Result: CSR arrays LV, JC, IA
$LV \leftarrow []$ /* Array of non-zero entries */
$JC \leftarrow []$ /* Array of column indices */
$IA \leftarrow [0]$ /* Row indices */
$nz \leftarrow 0$ /* Counter of non-zero entries */
for $i \leftarrow 0$ **to** $m - 1$ **do**
  for $j \leftarrow 0$ **to** $n - 1$ **do**
    if $A[i, j] \neq 0$ **then**
      $LV.append(A[i, j])$
      $JC.append(j)$
      $nz \leftarrow nz + 1$
    end
  end
  $IA.append(nz)$ /* Index where next row begins */
end
return LV, JC, IA

We then implemented a custom function implementing the matrix-vector product, reducing the complexity from $O(V^2)$ to $O(E)$.

Algorithm 5: CSR Matrix-Vector Product

Data: CSR arrays (LV, JC, IA), current rank vector $\mathbf{x}_k$, dangling node indices D
Result: Intermediate rank vector $\mathbf{w}$
$n \leftarrow$ length of $\mathbf{x}_k$
$\mathbf{w} \leftarrow \mathbf{0}_{n \times 1}$ /* Initialize result vector with zeros */
/* Step 1: Sparse Matrix Multiplication */
for $i \leftarrow 0$ **to** $n - 1$ **do**
  for $k \leftarrow IA[i]$ **to** $IA[i + 1] - 1$ **do**
    $\mathbf{w}[i] \leftarrow \mathbf{w}[i] + LV[k] \times \mathbf{x}_k[JC[k]]$ /* Only non-zero links */
  end
end
/* Step 2: Dangling Node Contribution */
$dangling_sum \leftarrow \sum_{j \in D} \mathbf{x}_k[j]$ /* Sum of rank in dead-end nodes */
for $i \leftarrow 0$ **to** $n - 1$ **do**
  $\mathbf{w}[i] \leftarrow \mathbf{w}[i] + \frac{dangling_sum}{n}$ /* Distribute lost rank equally */
end
return $\mathbf{w}$

5 Final Algorithm

Algorithm 6: Advanced Power Method (CSR Implementation)

Data: LV, JC, IA, N, dangling node indices D, max iterations max_iter, damping factor m, tolerance tol

Result: Final rank vector $\mathbf{x}_k$

$\mathbf{x}_k \leftarrow \mathbf{1}_{N\times 1}$ `/* Initialize rank vector with ones */`

$eig \leftarrow \infty$

$eig_{old} \leftarrow -\infty$

while $max_iter > 0$ ***and*** $|eig - eig_{old}| > tol$ **do**

  $eig_{old} \leftarrow eig$

  `/* Perform the CSR vector matrix multiplication */`

  $intermediate_result \leftarrow cvm(LV, JC, IA, \mathbf{x}_k, D)$

  `/* Apply damping factor and teleportation */`

  $\mathbf{x}_k \leftarrow (1-m)(intermediate_result) + m\frac{\mathbf{1}}{\mathbf{n}}$

  `/* Normalize the vector to ensure it remains a probability distribution */`

  $\mathbf{x}_k \leftarrow \mathbf{x}_k / \sum x_k$

  `/* Update eigenvalue estimate using Rayleigh-like quotient */`

  $eig \leftarrow \mathbf{x}_k^T A \mathbf{x}_k$

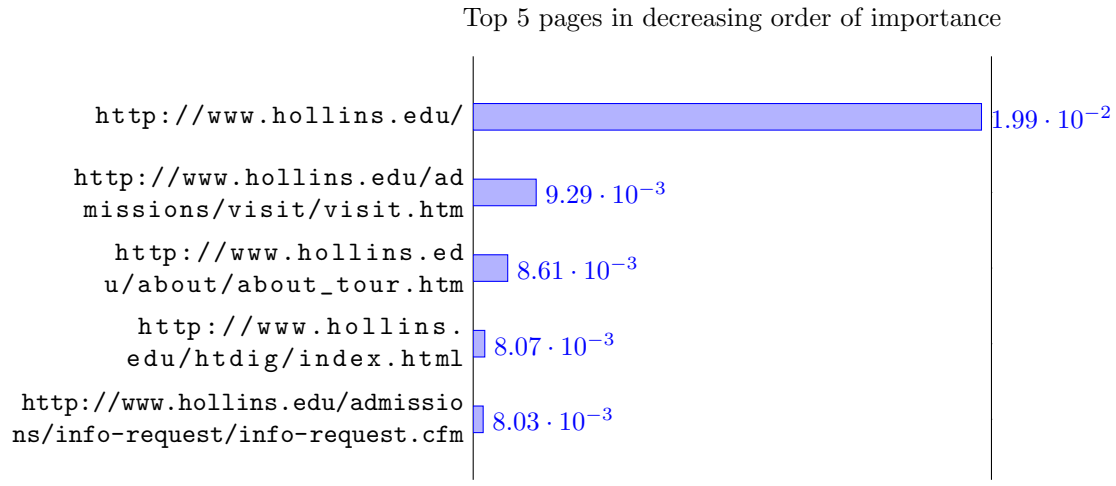
  $max_iter \leftarrow max_iter - 1$

end

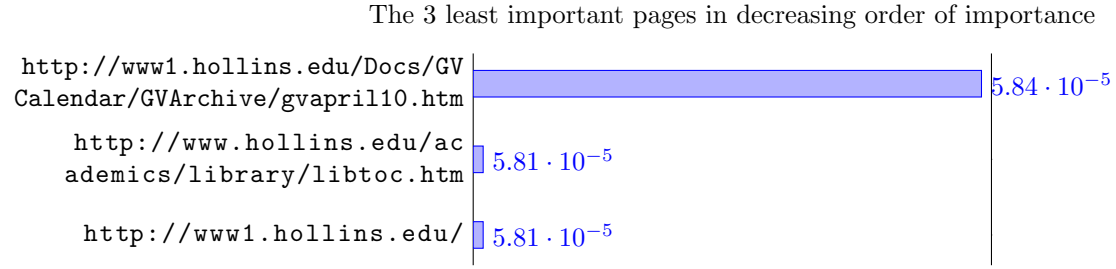
return $\mathbf{x}_k$

6 Results and Observations

6.1 Final Ranking



Running our final algorithm on the hollins.dat file, we observe that the most important page is the homepage of the website. The next most important page is less than half as important as the homepage. Consistent with the fact that with most websites you can always find links to pages such the homepage and the about-us page in the footer of almost every page, and that the homepage is almost always accessible from every other page, usually through an icon.



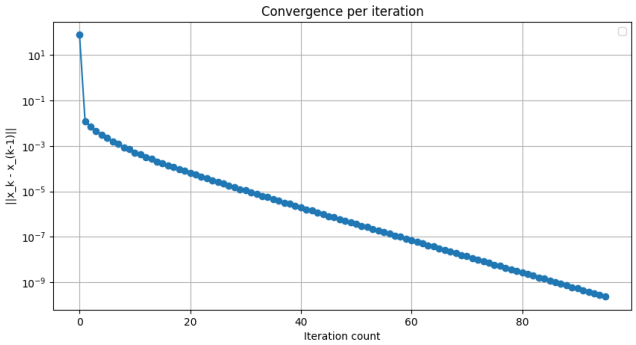
Conversely, apart from the 2 least important pages (both of which have 0 incoming links to them), the majority of the other least important pages are all archived pages. Therefore, we can conclude that the algorithm is working as intended.

6.2 Convergence

As we can see from the table and the plot below, the 2 norm of the difference of two sequential iterates x_{k-1} and x_k decreases drastically and converges to zero. This means that after enough iterations x_k stabilizes to a given value and the difference between two steps falls below a given threshold ($1e-12$). In the third column of the table we can also see that the ratio $\frac{\|x_k - x_{k-1}\|_2}{\|x_{k-1} - x_{k-2}\|_2}$ converges to a value that is close, but still smaller than $1 - m = 0.85$. This indicates that, accordingly to what we would expect, the second largest eigenvalue of our Pagerank matrix $|\lambda_2| \leq 0.85$.

k	$\|x_k - x_{k-1}\|_2$	$\frac{\|x_k - x_{k-1}\|_2}{\|x_{k-1} - x_{k-2}\|_2}$
1	7.75e+01	nan
5	3.09e-03	0.704
10	7.12e-04	0.833
20	7.89e-05	0.840
30	1.26e-05	0.814
40	2.26e-06	0.809
50	4.23e-07	0.810
60	8.11e-08	0.813
70	1.58e-08	0.816
80	3.08e-09	0.820
90	6.05e-10	0.822

(a) Measured residuals and ratios



(b) Convergence rate